

1. Backward

In **Backward Compatibility**, the goal is to ensure that **newer consumers can read older data**. This is the default and most common mode in Kafka because it allows you to update your downstream applications (consumers) at your own pace.

To keep a schema backward compatible, you generally follow these two rules:

1. **Only delete fields.**
2. **Only add optional fields** (fields that have a **default value**).

Example 1: Adding an Optional Field (Most Common)

Imagine you are tracking movie ratings. You want to start tracking which "platform" (iOS, Android, Web) a rating came from, but you don't want to break your old reporting tools.

Version 1 (Old):

```
{  
  "type": "record",  
  "name": "Rating",  
  "fields": [  
    {"name": "movieId", "type": "int"},  
    {"name": "rating", "type": "float"}  
  ]  
}
```

Version 2 (New - with Default):

```
{  
  "type": "record",  
  "name": "Rating",  
  "fields": [  
    {"name": "movieId", "type": "int"},  
    {"name": "platform", "type": "string", "default": "iOS"}  
  ]  
}
```

```
{ "name": "rating", "type": "float"},  
  
  { "name": "platform", "type": "string", "default": "unknown"}  
  
]  
}
```

If a consumer using Version 2 reads an **old** Version 1 message, it sees that the `platform` field is missing and automatically fills it with the value `"unknown"`.

Example 2: Deleting a Field

You decide no longer want to collect "middle names" in a user profile to simplify your database.

Version 1 (Old):

```
{  
  
  "type": "record",  
  
  "name": "User",  
  
  "fields": [  
  
    { "name": "id", "type": "int"},  
  
    { "name": "first_name", "type": "string"},  
  
    { "name": "middle_name", "type": "string"}  
  
  ]  
}
```

Version 2 (New):

```
{  
  
  "type": "record",  
  
  "name": "User",  
  
  "fields": [  
  
    { "name": "id", "type": "int"},  
  
    { "name": "first_name", "type": "string"},  
  
    { "name": "last_name", "type": "string"}  
  
  ]  
}
```

```
{  
  "name": "first_name", "type": "string"  
}  
]  
}
```

When a consumer using Version 2 reads an old message (which has a middle name), it simply ignores the `middle_name` field because it is no longer part of its schema definition.

Compatibility Flow

When using **Backward** mode, you should follow this upgrade order:

1. **Upgrade Consumers first:** They will now be able to handle both old and new data formats.
2. **Upgrade Producers second:** They can then start sending the new data format.

Summary Table for Backward Mode

Action	Allowed Backward?	Reason
Add Optional Field	Yes	Consumer uses the default value for old messages.
Add Required Field	No	Consumer won't know what value to use for old messages.
Delete Any Field	Yes	Consumer simply ignores the data in old messages.

2. Forward

In **Forward Compatibility**, the rule is that **older consumers must be able to read newer data**. This is useful when you want to upgrade your Producers first and don't want to wait for every downstream Consumer to be updated.

Example: Adding a Field (Forward Compatible)

Imagine you are a social media app. You want to start sending a "location" tag with every post, but you have thousands of different consumer apps (analytics, notifications, logs) that aren't ready to use it yet.

Version 1 (Old - currently in use by Consumers):

```
{  
  "type": "record",  
  "name": "Post",  
  "fields": [  
    {"name": "userId", "type": "int"},  
    {"name": "content", "type": "string"}  
  ]  
}
```

Version 2 (New - produced by the New Producer):

```
{  
  "type": "record",  
  "name": "Post",  
  "fields": [  
    {"name": "userId", "type": "int"},  
    {"name": "content", "type": "string"},  
    {"name": "location", "type": "string"}  
  ]  
}
```

```
]
}
```

When the **Old Consumer** (V1) receives a **New Message** (V2), it sees the `userId` and `content` it expects and simply **ignores** the `location` field. It doesn't crash; it just takes what it needs.

Comparison: Backward vs. Forward

Feature	Backward Compatibility	Forward Compatibility
Focus	New Consumer, Old Data	Old Consumer, New Data
Key Rule 1	Add fields with Defaults	Add Any fields
Key Rule 2	Delete Any fields	Delete fields with Defaults
Upgrade Order	Consumers first	Producers first

Summary Table: What can you do?

Action	Backward	Forward	Full
Add field with default	yes	yes	yes
Add required field	no	yes	no
Delete field with default	yes	yes	yes
Delete required field	yes	no	no

3. Full

Full Compatibility ensures that you are safe in both directions. It means that old consumers can read new data AND new consumers can read old data. In this mode, you can upgrade your Producers and Consumers in any order without worrying about breaking the pipeline.

The Rules of Full Compatibility

To achieve this, the Schema Registry is very strict.

1. Adding a field: You can only add a field if it has a default value.
2. Deleting a field: You can only delete a field if it already had a default value.

Example: The "Safe" Evolution

Imagine a banking transaction system where uptime is critical and you cannot coordinate team schedules to upgrade everyone at once.

Version 1 (Initial):

```
{  
  "type": "record",  
  "name": "Transaction",  
  "fields": [  
    {"name": "id", "type": "string"},  
    {"name": "amount", "type": "double"}  
  ]  
}
```

Version 2 (Full Compatible Update):

We want to add a currency field. To keep it Full, we MUST provide a default.

```
{  
  "type": "record",  
  "name": "Transaction",
```

```
"fields": [  
  {"name": "id", "type": "string"},  
  {"name": "amount", "type": "double"},  
  {"name": "currency", "type": "string", "default": "USD"}  
]  
}
```

Why this is "Full":

- Backward check: A new consumer (V2) reading old data (V1) sees no `currency` and fills it with "USD". (Success)
- Forward check: An old consumer (V1) reading new data (V2) sees the extra `currency` field and ignores it. (Success)

Comparison of the Three Main Modes

Action	Backward	Forward	Full
Add field with default	yes	yes	yes
Add required field	no	yes	no
Delete required field	yes	no	no
Delete field with default	yes	yes	yes
Upgrade Order	Consumers First	Producers First	Any Order

When to use Full Compatibility?

Use Full Compatibility for your most important production topics where:

- Many different teams (who don't talk to each other) consume the same data.
- You want the maximum flexibility for deployments.
- You want to avoid "stop-the-world" migrations where everything has to be updated at exactly the same time.

4. Code example

To pull schemas dynamically, we use the `client.get_latest_version()` or `client.get_version()` methods. This allows the Producer and Consumer to fetch the actual schema definitions from the Registry at runtime.

Scenario Recap

- **Producer:** Fetches **Version 1** from the Registry.
- **Consumer:** Fetches **Version 2** (the latest) from the Registry.
- **Compatibility:** BACKWARD.

1. The Producer (Pulls V1)

This Producer specifically asks for **Version 1** to simulate a legacy system that hasn't been updated yet.

```
from confluent_kafka import Producer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroSerializer
from confluent_kafka.serialization import StringSerializer, SerializationContext, MessageField
```

```
# Setup Client
```

```
sr_client = SchemaRegistryClient({'url': 'http://localhost:8081'})
subject = 'user-updates-value' #topic name - value
```

```
# PULL SCHEMA VERSION 1 FROM REGISTRY
```

```
v1_meta = sr_client.get_version(subject, version=1)
v1_schema_str = v1_meta.schema.schema_str
print(v1_schema_str)
print(f"Producer pulled Schema Version: {v1_meta.version}")
```

```
# Setup Producer
```

```
producer = Producer({'bootstrap.servers': 'localhost:9092'})
avro_serializer = AvroSerializer(sr_client, v1_schema_str)
string_serializer = StringSerializer('utf_8')
```

```
# Data matching V1 (id, name)
```

```
user_data = {"id": 101, "name": "Alice"}
```

```
producer.produce(
    topic='user-updates',
    key=string_serializer('alice-key'),
    value=avro_serializer(user_data, SerializationContext('user-updates', MessageField.VALUE))
```



```
)
```

```
producer.flush()
print("Sent V1 message to Kafka.")
```

2. The Consumer (Pulls Latest / V2)

The Consumer always pulls the **latest** schema. Because the mode is BACKWARD, it can use this new schema to read the old data produced above.

```
from confluent_kafka import Consumer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroDeserializer
from confluent_kafka.serialization import SerializationContext, MessageField

# Setup Client
sr_client = SchemaRegistryClient({'url': 'http://localhost:8081'})
subject = 'user-updates-value'

# PULL THE LATEST SCHEMA (Version 2) FROM REGISTRY
latest_meta = sr_client.get_latest_version(subject)
v2_schema_str = latest_meta.schema.schema_str

print(f"Consumer pulled Latest Schema Version: {latest_meta.version}")

# Setup Consumer
consumer = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'backward-test-group',
    'auto.offset.reset': 'earliest'
})
consumer.subscribe(['user-updates'])

# The Deserializer uses V2 to read whatever comes from the wire
avro_deserializer = AvroDeserializer(sr_client, v2_schema_str)

try:
    while True:
        msg = consumer.poll(1.0)
        if msg is None: continue

        # Deserialization magic happens here
        user = avro_deserializer(msg.value(), SerializationContext(msg.topic(), MessageField.VALUE))
```

```
print(f"Consumed Data: {user}")  
# If V1 was sent, 'region' will appear here with its default value from V2
```

```
except KeyboardInterrupt:  
    pass  
finally:  
    consumer.close()
```

Why this is powerful

By pulling from the registry:

1. **Centralized Truth:** You don't have to copy-paste JSON strings into your code.
2. **Version Control:** The Producer can stay on version 1 safely while the rest of the ecosystem moves to version 10.
3. **Automatic Mapping:** The `AvroDeserializer` automatically handles the "translation" between the schema ID embedded in the message (V1) and the schema you provided to the constructor (V2).

Troubleshooting Tip

If `sr_client.get_version(subject, version=1)` fails, it means you haven't registered the schemas yet. You can register them via `curl` or using `sr_client.register_group()` before running these scripts.